Walker Tienkung: Complete Development Guide

This guide covers camera setup, hand mimicry, and voice/audio interaction for the Walker Tienkung robot.

Part A: Camera & Hand Mimic Setup

Phase 1: Hardware Connection (CRITICAL)

Due to the high bandwidth requirements of raw video streaming, **Wi-Fi is not reliable** for video. You must use a wired Ethernet connection for development.

1. **Connect Cable:** Plug an Ethernet cable from your PC's Ethernet port to the **Robot's Debugging Port** (located on the back).
2. **Configure PC Network:**
   - Go to **Network Settings** -> **Ethernet** -> **IPv4 Settings**.
   - Set **Method** to `Manual`.
   - **IP Address:** `192.168.41.100` (Any IP ending in 10-250 is fine, except 1, 2, or 3).
   - **Netmask:** `255.255.255.0`.
   - **Gateway:** Leave blank.
   - **Save** and Apply.

Phase 2: Establish Connection

1. **Verify Connection:** Open a terminal on your PC and ping the Orin board (where the camera lives).

   ```
   ping 192.168.41.2
   ```

   If you get a reply, you are connected.
2. **SSH into the Robot (Orin Board):**

   ```
   ssh nvidia@192.168.41.2
   ```

   (Password is usually `nvidia`)

Phase 3: Start the Camera Driver

Once inside the robot (SSH terminal), you must launch the camera driver.

1. **Launch the Driver:**

   ```
   cd ~/orbbec_camera_ros2
   source install/setup.bash
   ros2 launch orbbec_camera gemini_330_series.launch.py
   ```

Keep this terminal open and running.

2. **Verify Topics (New Terminal):** Open a **second** SSH terminal (`ssh nvidia@192.168.41.2`) and check if the camera is publishing:

```
ros2 topic list
```

You must see `/camera/color/image_raw` in the list.

Phase 4: Install Dependencies (Hand Mimic)

In the **second** SSH terminal, install the Python libraries required for hand tracking.

1. **Install Libraries:**

```
pip install opencv-python mediapipe rclpy
```

Note: If `rclpy` fails, it is likely already installed via ROS. You can ignore it.

Phase 5: Create the Hand Mimic Script

Create the Python script that listens to the camera and controls the hands.

1. **Create File:**

```
nano mimic_dual_hands.py
```

2. **Paste Code:** (Use the full `mimic_dual_hands.py` code provided in the previous response).
3. **Run the Script:**

```
python3 mimic_dual_hands.py
```

Part B: Voice, TTS, and Audio

The robot has a built-in speaker and microphone array managed by the **Orin Board** (`192.168.41.2`). You can control it via ROS 2 topics.

1. Audio Architecture

- **Hardware:** Microphone & Speaker connected to Orin Board.
- **Service Node:** `xunfei_dev_socket` (Must be running for voice to work).
- **Audio Storage Path:** `/home/nvidia/data/speech/` (on Orin).

2. Starting the Voice Service

Before using any voice features, you must ensure the voice driver is running.

1. **SSH into Orin:** `ssh nvidia@192.168.41.2`

2. **Launch Service:**

```
source ~/audio_ros2/install/setup.bash
ros2 launch xunfei_dev_socket xunfei_dev_all.launch.py
```

Keep this terminal open.

3. Text-to-Speech (TTS) - The "Repeater"

The robot does not have a direct "Say this text" topic. It only plays files. To make it speak text, we generate a temporary MP3 file using Python and play it.

Prerequisites:

```
pip install gTTS
```

Script: `repeater_force.py` **(Keeps robot awake & Repeats you)** This script forces the microphone to stay open (preventing sleep) and repeats what you say.

1. **Create File:** `nano repeater_force.py`
2. **Paste Code:**

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import json
import os
from gtts import gTTS

class VoiceRepeaterForce(Node):
    def __init__(self):
        super().__init__('voice_repeater_force')

        # --- PART 1: KEEP AWAKE (The "Force" Logic) ---
        # Publisher to send commands TO the robot
        self.cmd_pub = self.create_publisher(String, '/xunfei/aiui_msg', 10)

        # Timer: Trigger the wake-up command every 4.0 seconds
        self.timer = self.create_timer(4.0, self.force_wake_callback)

        # --- PART 2: REPEATER (The "Echo" Logic) ---
        # Subscriber: Listen to what the robot hears
        self.subscription = self.create_subscription(
            String,
            '/xunfei/aiui_msg',
            self.asr_callback,
            10
        )

        # Publisher: Play audio files
        self.audio_pub = self.create_publisher(String, '/xunfei/tts_play', 10)

        self.last_text = ""
        self.get_logger().info('>>> FORCE REPEATER RUNNING: Keeping awake & listening...'
```

```
    def force_wake_callback(self):
        """Sends the Wake Up event (Event 4) periodically"""
        msg = String()
        # This JSON command forces the microphone to open
        msg.data = '{"type":"aiui_event","content":{"eventType":4,"arg1":1,"arg2":0}}'
        self.cmd_pub.publish(msg)

    def asr_callback(self, msg):
        """Processes text coming from the robot"""
        try:
            data = json.loads(msg.data)

            # We only care about Event 1 (Result)
            if "content" in data and "result" in data["content"]:
                result = data["content"]["result"]

                if "text" in result and "ws" in result["text"]:
                    words = result["text"]["ws"]
                    sentence = ""

                    for item in words:
                        if "cw" in item and len(item["cw"]) > 0:
                            sentence += item["cw"][0]["w"]

                    # If valid sentence and not a duplicate
                    if sentence and len(sentence) > 1 and sentence != self.last_text:
                        self.get_logger().info(f'I heard: "{sentence}"')
                        self.last_text = sentence
                        self.repeat_text(sentence)

        except Exception:
            pass

    def repeat_text(self, text):
        """Converts text to MP3 and plays it"""
        try:
            file_path = "/home/nvidia/data/speech/repeat_temp.mp3"

            # Generate English Audio (Change 'en' to 'zh' for Chinese)
            tts = gTTS(text=text, lang='en')
            tts.save(file_path)

            # Play Audio
            msg = String()
            payload = {"file": file_path}
            msg.data = json.dumps(payload)
            self.audio_pub.publish(msg)
            self.get_logger().info(f'Repeating: "{text}"')

        except Exception as e:
            self.get_logger().error(f"Audio Error: {e}")

def main(args=None):
    rclpy.init(args=args)
    node = VoiceRepeaterForce()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

3. **Run It:** `python3 repeater_force.py`

4. Playing Custom Audio Files

If you just want to play a specific MP3 file (e.g., for a dance routine):

1. **Transfer File:** Send your MP3 to the robot.

```
# From PC
scp my_song.mp3 nvidia@192.168.41.2:/home/nvidia/data/speech/
```

2. **Play Command (ROS 2):**

```
ros2 topic pub --once /xunfei/tts_play std_msgs/msg/String "{data: '{\"file\": \"/hom
```

3. **Stop Command:**

```
ros2 topic pub --once /xunfei/tts_play std_msgs/msg/String "{data: '{\"cmd\": \"stop\
```